Benchmarking, Predicate Pushdown y Optimización Columnar

Esteban Valenzuela

Carrera en Ciencia de Datos e Inteligencia Artificial, Facultad de Ingeniería en Sistemas, Escuela Politécnica Nacional, Quito, Ecuador.

Contributing authors: esteban.valenzuela@epn.edu.ec;

Abstract

Este informe evalúa empíricamente el impacto del cuello de botella de Entrada/Salida (I/O) al comparar CSV, un formato basado en filas, con Apache Parquet, un formato columnar, en dos ejecuciones secuenciales de un benchmark de transacciones sintéticas de retail: 5 millones y 15 millones de registros. Se comparan escritura, lectura completa, consultas con proyección de columnas, filtrado y agregación. En la ejecución de 15 millones de registros, el consumo de RAM reportado aumentó de 2.0 GB a 4.9 GB respecto de la ejecución de 5 millones. Parquet ocupó 225.39 MB frente a 740.53 MB en CSV, una reducción de 69.56 %, y su escritura fue 8.71 veces más rápida. Para la agregación selectiva, Parquet obtuvo una aceleración de 18.53 veces. En la consulta avanzada con los predicados "Juguetes" y "Completado", el tiempo medido con Parquet fue 4.14 veces menor; sin embargo, los formatos devolvieron cantidades distintas de filas, por lo que esa comparación temporal no demuestra equivalencia semántica y debe interpretarse con cautela. Los resultados ilustran las ventajas de la proyección columnar y de la reducción de I/O, así como la importancia de validar que los motores ejecuten exactamente la misma consulta.

Keywords: Parquet, CSV, Predicate Pushdown, Column Pruning, I/O Bottleneck, Benchmarking

1 Introducción

Las organizaciones enfrentan volúmenes de datos crecientes y, con ellos, el desafío de reducir el **cuello de botella de Entrada/Salida (I/O)** entre almacenamiento y memoria. En formatos basados en filas como **CSV**, una consulta selectiva suele

requerir leer y deserializar registros completos, incluso cuando solo se necesitan unos pocos campos. **Apache Parquet** organiza los datos por columnas y permite aplicar **proyección de columnas (Column Pruning)**; además, sus metadatos estadísticos pueden permitir que un motor descarte **Row Groups** que no satisfacen un predicado antes de leer sus datos. La eficacia de este descarte depende de las estadísticas disponibles y de la distribución de los datos. La compresión por bloques **Snappy** reduce el volumen almacenado y transferido, a cambio del trabajo de descompresión durante la lectura.

Este laboratorio compara empíricamente **CSV y Parquet** en dos ejecuciones secuenciales de un benchmark de transacciones sintéticas de retail, con **5 millones y 15 millones de registros**. Se analizan escritura, tamaño de archivo, lectura completa, consultas selectivas, filtros y agregaciones, con énfasis en la evolución del rendimiento al triplicar el volumen y en la interpretación de una consulta avanzada de negocio con múltiples predicados.

2 Secciones de trabajo técnico

2.1 Comparación de métricas empíricas

La tabla de comparación presenta los resultados reportados para ambas ejecuciones. Las **aceleraciones** son cocientes entre el tiempo de CSV y el de Parquet; por tanto, un valor superior a uno favorece a Parquet. Las reducciones de almacenamiento se calculan respecto del tamaño CSV. Los resultados de escritura y lectura completa corresponden a operaciones sobre archivos, mientras que los tiempos de consultas corresponden a las operaciones analíticas reportadas en el laboratorio.

Métrica	**Ejecución 1 (5M)** *CSV / Parquet*	**Ejecución 2 (15M)** *CSV / Parquet*	**Ventaja de Parquet**
RAM (creación)	2.0 GB	4.9 GB	Aumentó 2.45× al triplicar los registros.
Escritura	23.61 / 2.16 s	72.92 / 8.37 s	Parquet fue 10.93× y 8.71× más rápido.
Tamaño en disco	244.55 / 75.14 MB	740.53 / 225.39 MB	Parquet redujo el espacio 69.28% y 69.56%.
Lectura completa	3.19 / 0.91 s	13.92 / 3.02 s	Parquet fue 3.51× y 4.61× más rápido.
Agregación selectiva	2.29 / 0.088 s	8.04 / 0.434 s	Parquet fue 26.02× y 18.53× más rápido.
Filtrado	3.95 / 0.66 s	13.87 / 3.35 s	5M: 5.94×. 15M: 4.14× observado, con predicados distintos.
GroupBy	5.54 / 0.74 s	13.62 / 3.24 s	Parquet fue 7.54× y 4.20× más rápido.

Tabla 1. Comparación del rendimiento (5M y 15M de registros)

Al triplicar los registros, el tamaño de ambos archivos creció aproximadamente tres veces, mientras que la **RAM reportada pasó de 2.0 GB a 4.9 GB**. La escritura y lectura de CSV escalaron con un aumento más pronunciado que el volumen en varias mediciones. Las aceleraciones deben entenderse como resultados de este entorno y de estas operaciones, no como constantes universales: **caché del sistema**, almacenamiento, configuración del escritor, particionamiento y estado de la sesión pueden afectar las mediciones.

2.2 Reto 1: Rendimiento con 15 millones de registros

El paso de **5M a 15M de registros** elevó la RAM del sistema reportada durante la creación de **2.0 GB a 4.9 GB**, un incremento de 2.9 GB (2.45 veces). La Figura 5 deja la lectura de línea base de 5M en evidencias; la Figura 1 muestra directamente la ejecución de 15M, incluyendo los 15,000,000 de registros, 572.2 MB de uso del DataFrame y el indicador del sistema de 4.9/12.7 GB. El consumo del DataFrame y el del sistema son indicadores distintos y no deben confundirse.

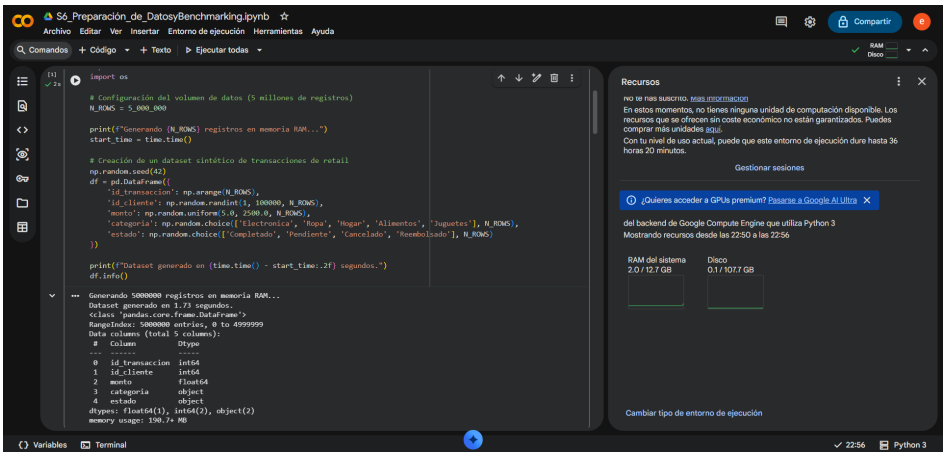


Fig. 1 Ejecución de 15M: creación de 15,000,000 de registros, uso de 572.2 MB por el DataFrame e indicador de RAM del sistema de 4.9/12.7 GB.

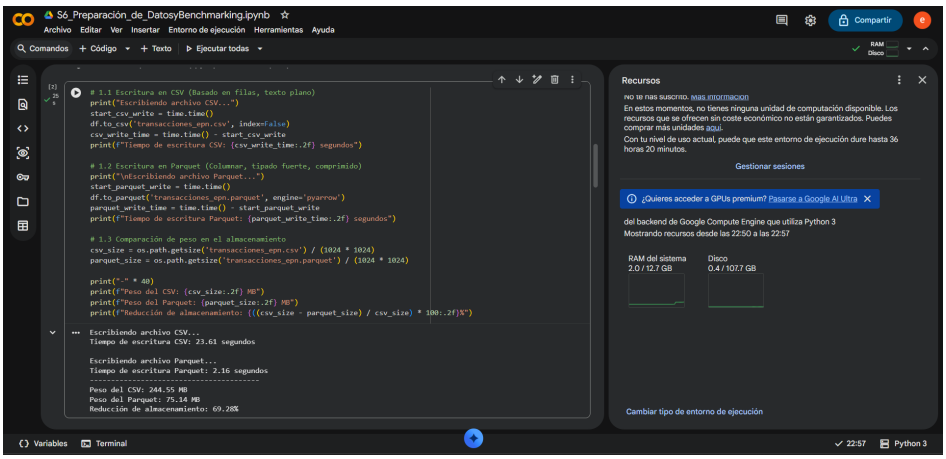


Fig. 2 Ejecución de 15M: escritura de CSV y Parquet, tamaños de archivo y reducción de almacenamiento del 69.56%.

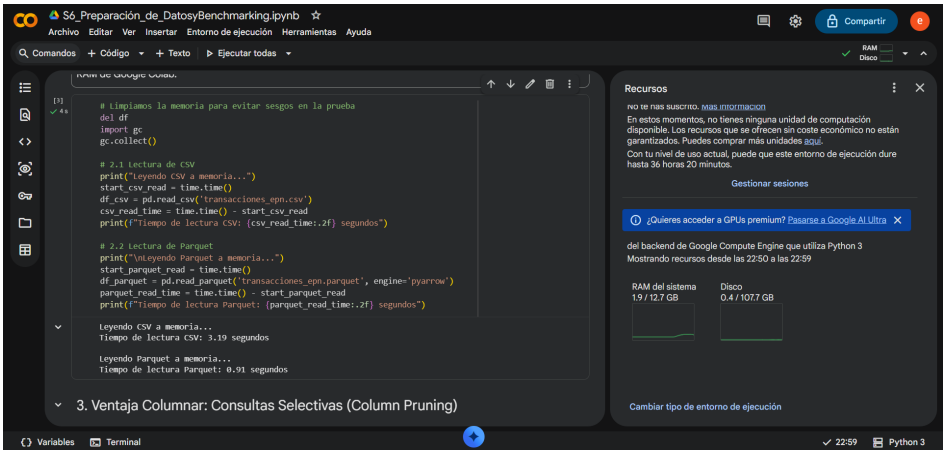


Fig. 3 Ejecución de 15M: lectura completa de CSV en 13.92 s y de Parquet en 3.02 s.

Estas mediciones muestran que el archivo Parquet fue menor y las operaciones reportadas fueron más rápidas en las dos escalas. El incremento de los tiempos de escritura y lectura al triplicar los datos refleja mayor trabajo de I/O, mientras que la reducción de bytes almacenados en Parquet mitiga ese costo. La medición de memoria documenta el consumo observado, pero no es un perfil del **recolector de basura (Garbage Collector, GC)**; por ello, el posible efecto de objetos temporales y de la deserialización de texto es una explicación técnica, no una causa cuantificada en este experimento.

El aumento de volumen incrementa la cantidad de objetos temporales y referencias que el proceso debe gestionar. En una ruta basada en **CSV**, la lectura involucra

interpretar texto delimitado, convertir los valores a sus tipos de destino y construir estructuras en memoria. Esta **deserialización de texto** y las asignaciones asociadas pueden generar presión adicional de memoria y trabajo para el **GC**. Sin métricas específicas del GC, no es posible afirmar que este haya sido el factor dominante ni estimar pausas atribuibles a él.

Parquet en la ejecución de 15M, ocupó 225.39 MB frente a 740.53 MB de CSV (**69.56% menos**), como se observa en la Figura 2; para 5M, la Figura 6 registra una reducción de 69.28%. Menos datos transferidos disminuyen la presión de **I/O**; al leer CSV, además, es necesario interpretar texto y convertirlo a los tipos de destino, mientras que Parquet permite leer datos tipados y comprimidos. La descompresión también consume CPU, de modo que la ventaja medida resulta del balance entre menor I/O y ese costo de procesamiento.

En las operaciones completas, **Parquet fue 3.51 veces más rápido** en lectura a 5M y **4.61 veces** a 15M, según las Figuras 7 y 3. Para escritura, las aceleraciones observadas fueron 10.93 y 8.71 veces, respectivamente, como se calcula a partir de las Figuras 6 y 2. La reducción de aceleración de escritura al escalar muestra que el beneficio relativo no se mantiene necesariamente constante: el tiempo Parquet creció 3.88 veces entre ejecuciones, y otros costos del entorno pueden adquirir mayor peso. En el caso de consultas que requieren pocas columnas, la **proyección de columnas** evita leer campos irrelevantes y refuerza la ventaja columnar: la agregación selectiva fue **26.02 veces más rápida a 5M y 18.53 veces a 15M**, conforme a las salidas reportadas para cada escala.

2.3 Reto 2: Predicate Pushdown y consulta de negocio

La Figura 4 documenta la consulta avanzada y sus salidas. En la captura se observa una diferencia concreta entre las operaciones: el código CSV aplica únicamente categoría **"Juguetes"**, mientras que la lectura Parquet aplica dos condiciones, categoría **"Juguetes"** y estado **"Completado"**. Por ello, las 3,999,407 filas de CSV y las 3,751,678 de Parquet no son resultados de consultas equivalentes. La diferencia de **247,729 filas** es coherente con la condición adicional de estado en Parquet (si ambas lecturas usan el mismo archivo y los mismos valores); no debe interpretarse como una discrepancia inexplicada del motor.

Los tiempos observados fueron 13.8722 s para CSV y 3.3476 s para Parquet, cuyo cociente es **4.14**. Sin embargo, como CSV evalúa un solo predicado y Parquet dos, esa aceleración no aísla el efecto de **Predicate Pushdown** ni constituye una comparación de rendimiento semánticamente equivalente. La evidencia disponible respalda los tiempos, los filtros implementados y los conteos tal como aparecen en la captura, pero no permite atribuir toda la diferencia temporal exclusivamente al formato. Para una comparación válida, se deben aplicar ambos predicados a CSV y Parquet sobre el mismo dataset y verificar que las cardinalidades coincidan.

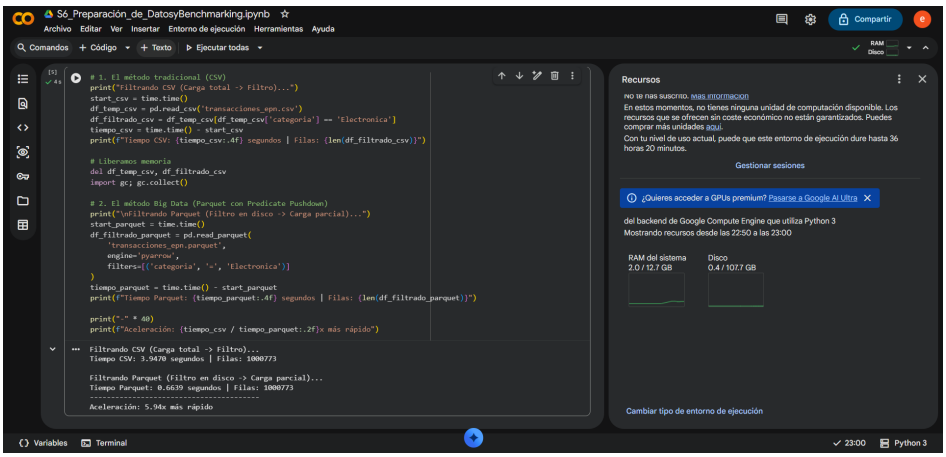


Fig. 4 Consulta avanzada reportada: el código CSV filtra solo por "Juguetes", mientras Parquet filtra por "Juguetes" y "Completado"; la captura registra tiempos, filas devueltas y 3.7/12.7 GB de RAM del sistema.

En una lectura **CSV** convencional, el lector recorre las filas y debe analizar los campos de texto para evaluar los predicados. Aunque después se proyecten solo ciertas columnas, CSV no ofrece por sí mismo estadísticas por bloques que permitan a un motor columnar omitir con seguridad rangos de filas antes de leerlos. Una implementación puede optimizar el análisis o la proyección, pero el formato sigue requiriendo recorrer el contenido textual para localizar y evaluar los valores pertinentes.

En **Parquet**, el lector puede combinar **proyección de columnas** con **Predicate Pushdown**. La proyección solicita únicamente los campos necesarios para filtrar y producir la salida. Además, el archivo se organiza en **Row Groups** y contiene metadatos, incluidas estadísticas por columna cuando están disponibles. Si las estadísticas de un Row Group demuestran que ningún registro puede satisfacer uno de los predicados, el motor puede descartarlo antes de leer y descomprimir sus páginas de datos. Para los grupos que no puedan descartarse, se leen las columnas pertinentes y se evalúan las condiciones; los metadatos no garantizan que todo grupo no coincidente sea omitido, en particular si los valores están distribuidos ampliamente o las estadísticas no son suficientemente selectivas.

Este mecanismo reduce **I/O** y la cantidad de datos que llegan a memoria, pero no equivale a afirmar que cualquier filtro sobre Parquet evita leer los registros: la poda depende de las estadísticas, el tamaño y orden de los **Row Groups**, la distribución de los valores y la ruta de consulta utilizada. En el laboratorio, el tiempo de Parquet fue menor, pero para atribuir con rigor toda la diferencia a **Predicate Pushdown** habría que verificar el plan y la configuración del lector, así como igualar el número de filas y las condiciones aplicadas en ambos formatos.

Para repetir el benchmark de forma comparable, se recomienda aplicar exactamente ambos predicados a los dos lectores; comprobar nombres y tipos de columnas, valores nulos y cadenas de filtro; comparar las filas resultantes o una huella determinista; y registrar la configuración de **Row Groups** y compresión. También conviene

repetir las mediciones con y sin caché, e informar el plan o los metadatos utilizados para la poda. Estas verificaciones permiten separar la mejora real de **I/O** de las diferencias causadas por la lógica de consulta.

2.4 Ejecuciones de los scripts en Google Colab

Las capturas de 5 millones se conservan aquí como evidencia de la línea base. Las capturas de creación, escritura y lectura de 15 millones se presentan junto al análisis del Reto 1 porque respaldan directamente sus métricas de RAM, I/O y tamaño. Las imágenes de Column Pruning y GroupBy que llevan nombres de ejecución 2 muestran resultados de 5M y se incluyen con pies que aclaran su contenido. La captura de la consulta avanzada permanece junto al análisis del Reto 2.

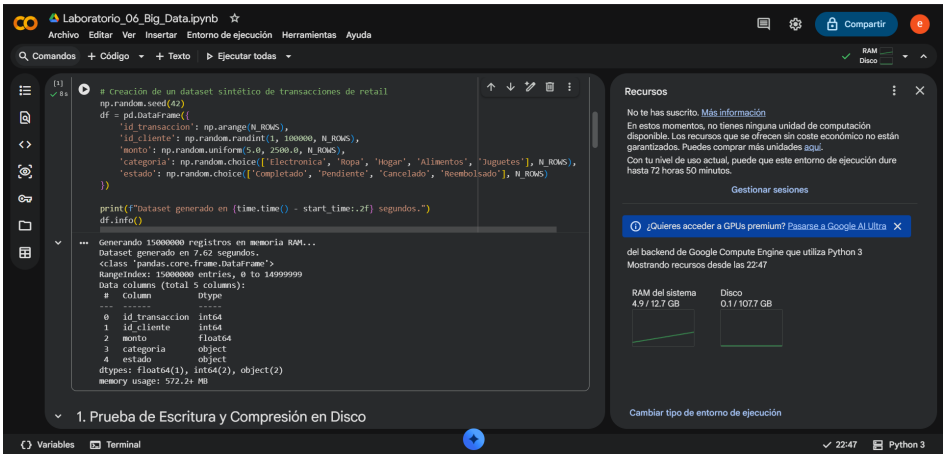


Fig. 5 Ejecución 1 (5M): creación del dataset sintético y recursos de memoria reportados.

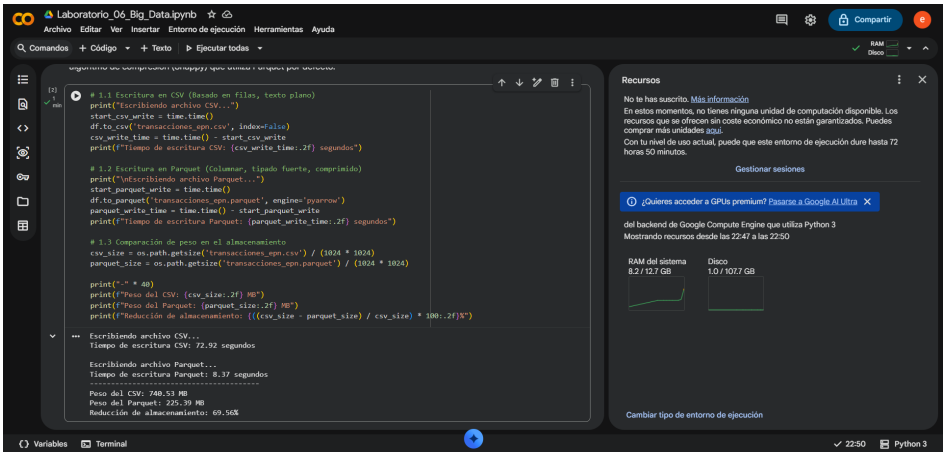


Fig. 6 Ejecución 1 (5M): comparación de escritura y tamaño de los archivos CSV y Parquet.

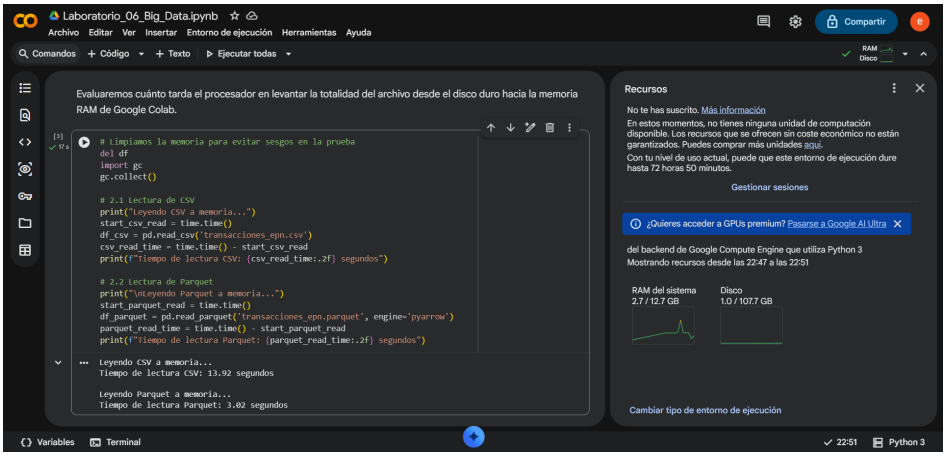


Fig. 7 Ejecución 1 (5M): tiempos de lectura completa para ambos formatos.

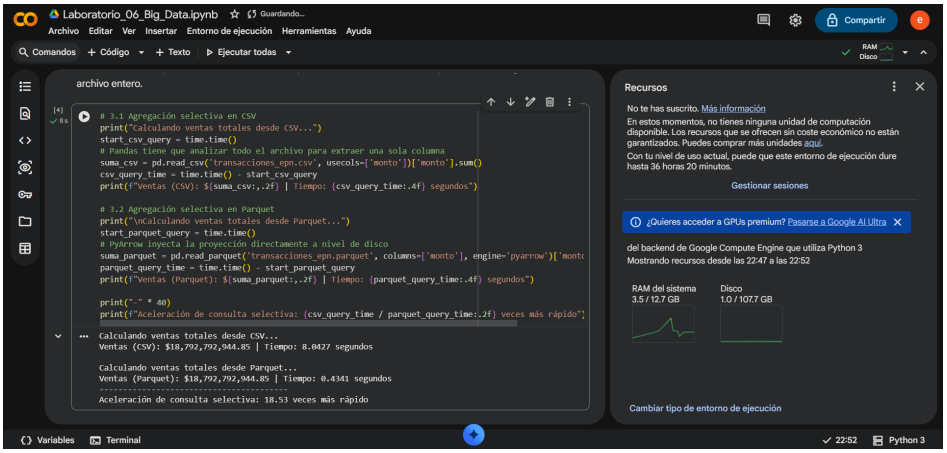


Fig. 8 Ejecución 1 (5M): prueba de agregación selectiva mediante proyección de columnas.

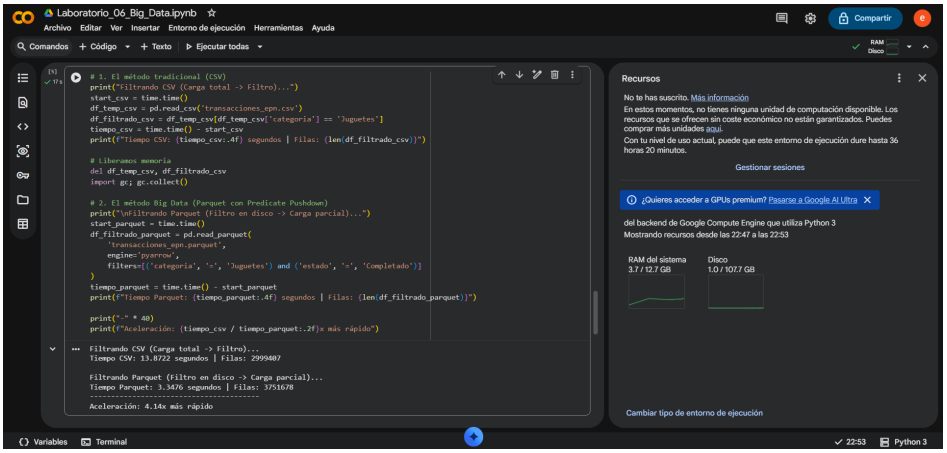


Fig. 9 Ejecución 1 (5M): filtro por categoría "Electronica" y comparación de tiempos y filas.

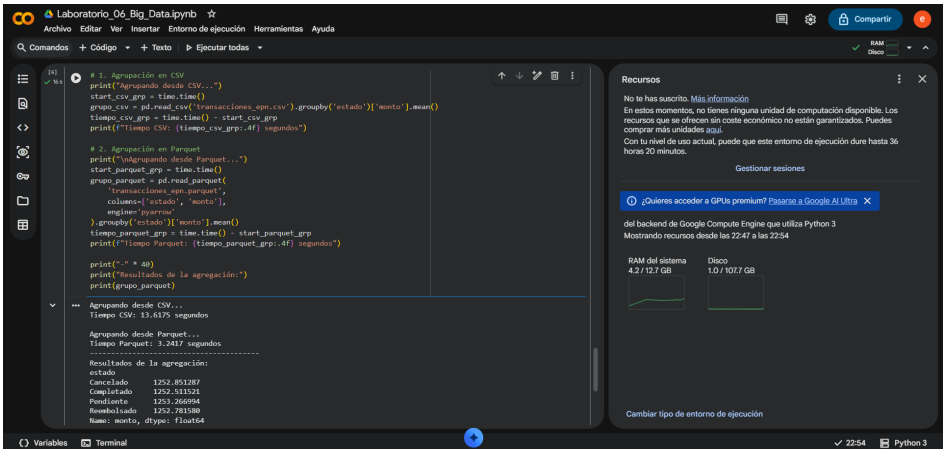


Fig. 10 Ejecución 1 (5M): resultados de la agregación GroupBy por estado.

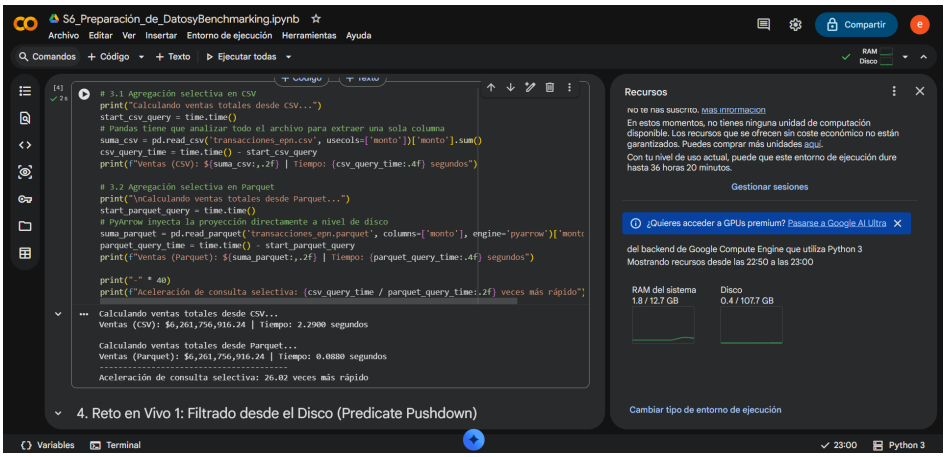


Fig. 11 Column Pruning (5M): aunque el archivo lleva el nombre de ejecución 2, la salida visible (2.2900 s y 0.0880 s) coincide con la línea base.

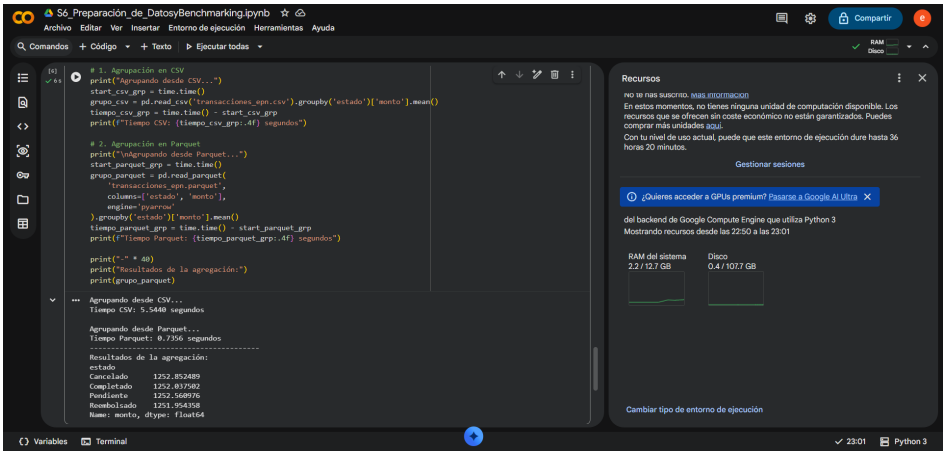


Fig. 12 GroupBy (5M): aunque el archivo lleva el nombre de ejecución 2, la salida visible (5.5440 s y 0.7356 s) coincide con la línea base.

3 Conclusiones

Los resultados muestran que, en el entorno evaluado, Apache Parquet ofrece ventajas consistentes frente a CSV para almacenamiento y análisis. Al escalar de 5M a 15M de registros, la RAM reportada pasó de 2.0 a 4.9 GB y el tamaño de archivo creció aproximadamente en proporción al volumen. Parquet redujo el almacenamiento en cerca del 69% en ambas ejecuciones y presentó menores tiempos de escritura, lectura completa, agregación selectiva y GroupBy.

La organización columnar permite que la proyección de columnas evite leer campos que no son necesarios. En consultas filtradas, los metadatos de Parquet pueden habilitar la poda de Row Groups, reduciendo I/O antes de cargar los datos; Snappy, por su parte, reduce bytes transferidos a cambio de trabajo de descompresión. Las mediciones de RAM no incluyeron perfiles específicos del GC, por lo que el posible efecto de la deserialización de texto y de las asignaciones debe interpretarse como explicación técnica plausible, no como una causalidad medida en este experimento.

El Reto 2 obtuvo una aceleración temporal observada de 4.14 veces para Parquet, pero devolvió una cantidad diferente de filas que CSV. Antes de concluir que ambos tiempos representan exactamente la misma consulta, es necesario revisar la equivalencia del filtrado y repetir la prueba con resultados validados. Con esa salvedad, el laboratorio evidencia que el formato columnar puede mitigar el carácter I/O Bound de las cargas analíticas, y que un benchmark confiable debe controlar tanto el rendimiento como la corrección semántica.

4 Declaración de uso de IA

Se utilizó Gemini 3 [1] como herramienta para facilitar el desarollo del informe. Específicamente, la IA se empleó para la estructuración de los datos presentados en la Tabla 1, el cálculo preciso de las diferencias en porcentajes comparativos entre formatos de archivos (CSV y Parquet), la adaptación del código LaTeX de las tablas al formato estándar de la plantilla de Springer, y la redacción clara de los pies de figura correspondientes. Todo el contenido generado, cálculos y código fueron revisados.

5 Repositorio Git Hub

https://github.com/EstebanValenzuela10/Laboratorio_06_Big_Data.git

Referencias

[1] Google: Gemini (1.5 Pro) [Large language model]. https://gemini.google.com (2026)

[2] David Galarza, Esteban Valenzuela: Notebook de experimentos y scripts de benchmark (CSV vs. Parquet). https://drive.google.com/file/d/1OP8LRwpe2LW-CaSOZCZWAyvTkns_q3Wv/view?usp=sharing (2026)